

	SR UNIVERSITY	
	Campus Warangal	
	Program: II - B. Tech (CS& AI) Batch:	
	Name: Pranjal Kumar	Reg No:2503a51663
	Department: Computer Science and AI	Semester: I
	AI Assisted Coding-Assignment	Date: 22/09/26

Problem 1

TASK 1 – Tree: Hospital Department Hierarchy

Prompt –

Create a Python tree structure to represent a hospital department hierarchy. Implement adding child nodes, displaying the hierarchy, searching for a department, and counting nodes. Explain how a tree is different from a list and provide test cases.

Output

```
Hospital Department Hierarchy:
Hospital
  Administration
  Medical
    Cardiology
    ICU
    Neurology

Searching for 'Cardiology':
Found: Cardiology

Total departments: 6

Running test cases...
All test cases passed.
PS C:\Users\DELL\OneDrive\Desktop\WTMP LAB> 
```

Code

Code

```
first.py

class DepartmentNode:
    def __init__(self, name):
        self.name = name
        self.children = []

    def add_child(self, child_name):
        child = DepartmentNode(child_name)
        self.children.append(child)
        return child

    def add_child_node(self, child_node):
        self.children.append(child_node)
        return child_node

    def display(self, level=0):
        print('  ' * level + self.name)
        for child in self.children:
            child.display(level + 1)

    def search(self, target_name):
        if self.name == target_name:
            return self

        for child in self.children:
            result = child.search(target_name)
            if result is not None:
                return result

        return None

    def count_nodes(self):
        total = 1
        for child in self.children:
            total += child.count_nodes()
        return total
```

Code

```
first.py

class HospitalDepartmentTree:
    def __init__(self, root_name):
        self.root = DepartmentNode(root_name)

    def add_department(self, parent_name, child_name):
        parent = self.root.search(parent_name)
        if parent is None:
            raise ValueError(f"Parent department '{parent_name}' not found.")
        return parent.add_child(child_name)

    def display_hierarchy(self):
        self.root.display()

    def search_department(self, department_name):
        return self.root.search(department_name)

    def count_departments(self):
        return self.root.count_nodes()

def test_hospital_tree():
    hospital = HospitalDepartmentTree("Hospital")

    # Add departments
    hospital.add_department("Hospital", "Administration")
    hospital.add_department("Hospital", "Medical")
    hospital.add_department("Medical", "Cardiology")
    hospital.add_department("Medical", "Neurology")
    hospital.add_department("Cardiology", "ICU")

    # Check total count
    assert hospital.count_departments() == 6, "Total department count should be 6"

    # Search for existing department
    found = hospital.search_department("Cardiology")
    assert found is not None and found.name == "Cardiology", "Cardiology should be found"

    # Search for non-existing department
    missing = hospital.search_department("Radiology")
    assert missing is None, "Radiology should not be found"

    # Check parent-child relationship
    parent = hospital.search_department("Medical")
    assert len(parent.children) == 2, "Medical should have 2 child departments"

    print("All test cases passed.")

if __name__ == "__main__":
    hospital = HospitalDepartmentTree("Hospital")
    hospital.add_department("Hospital", "Administration")
    hospital.add_department("Hospital", "Medical")
    hospital.add_department("Medical", "Cardiology")
    hospital.add_department("Medical", "Neurology")
    hospital.add_department("Cardiology", "ICU")

    print("Hospital Department Hierarchy:")
    hospital.display_hierarchy()

    print("\nSearching for 'Cardiology':")
    result = hospital.search_department("Cardiology")
    if result:
        print("Found:", result.name)
    else:
        print("Not found")

    print("\nTotal departments:", hospital.count_departments())

    print("\nRunning test cases...")
    test_hospital_tree()
```

Observation

This program creates a hierarchical tree of hospital departments where each department can have many child departments.

The root node is `Hospital`, and each child department is connected to its parent in a tree structure.

It supports adding departments, displaying the hierarchy, searching for a department, and counting total nodes.

Problem-2

Graph: Hospital Route Finder

Prompt

Write a Python program to represent hospital locations as a graph. Each location should be a node and each connection between two locations should be an edge. Use an adjacency list to store the graph. Add at least 5 hospital locations such as Reception, Pharmacy, ICU, Emergency, and Laboratory. Add connections between them, display the graph, check whether two locations are connected, and find a route between a source and destination using BFS or DFS. Also write test cases for at least 3 source-destination pairs

Code

```
first.py

class HospitalGraph:
    def __init__(self):
        self.graph = {}

    def add_location(self, location):
        if location not in self.graph:
            self.graph[location] = []

    def add_connection(self, location1, location2):
        if location1 not in self.graph:
            self.add_location(location1)
        if location2 not in self.graph:
            self.add_location(location2)

        if location2 not in self.graph[location1]:
            self.graph[location1].append(location2)
        if location1 not in self.graph[location2]:
            self.graph[location2].append(location1)

    def display_graph(self):
        print("Hospital Graph (Adjacency List):")
        for location in self.graph:
            print(f"{location} → {self.graph[location]}")

    def is_connected(self, source, destination):
        if source not in self.graph or destination not in self.graph:
            return False

        visited = set([source])
        queue = [source]

        while queue:
            current = queue.pop(0)
            if current == destination:
                return True

            for neighbor in self.graph[current]:
                if neighbor not in visited:
                    visited.add(neighbor)
                    queue.append(neighbor)

        return False

    def find_route(self, source, destination):
        if source not in self.graph or destination not in self.graph:
            return None

        visited = set([source])
        queue = [source]
        previous = {source: None}

        while queue:
            current = queue.pop(0)

            if current == destination:
                break

            for neighbor in self.graph[current]:
                if neighbor not in visited:
                    visited.add(neighbor)
                    previous[neighbor] = current
                    queue.append(neighbor)

        if destination not in previous:
            return None

        route = []
        current = destination
        while current is not None:
            route.append(current)
            current = previous[current]

        route.reverse()
        return route
```

Code

```
first.py

def test_hospital_graph():
    hospital = HospitalGraph()

    for loc in ["Reception", "Pharmacy", "ICU", "Emergency", "Laboratory"]:
        hospital.add_location(loc)

    hospital.add_connection("Reception", "Pharmacy")
    hospital.add_connection("Reception", "ICU")
    hospital.add_connection("Pharmacy", "Emergency")
    hospital.add_connection("ICU", "Emergency")
    hospital.add_connection("ICU", "Laboratory")
    hospital.add_connection("Emergency", "Laboratory")

    assert hospital.is_connected("Reception", "Laboratory") is True
    assert hospital.is_connected("Reception", "Emergency") is True
    assert hospital.is_connected("Reception", "X-Ray") is False

    route1 = hospital.find_route("Reception", "Laboratory")
    route2 = hospital.find_route("Pharmacy", "ICU")
    route3 = hospital.find_route("Emergency", "Reception")

    assert route1 == ["Reception", "ICU", "Laboratory"] or route1 == ["Reception", "ICU", "Emergency",
"Laboratory"]
    assert route2 is not None
    assert route3 is not None

    print("All graph tests passed.")

if __name__ == "__main__":
    hospital = HospitalGraph()

    for loc in ["Reception", "Pharmacy", "ICU", "Emergency", "Laboratory"]:
        hospital.add_location(loc)

    hospital.add_connection("Reception", "Pharmacy")
    hospital.add_connection("Reception", "ICU")
    hospital.add_connection("Pharmacy", "Emergency")
    hospital.add_connection("ICU", "Emergency")
    hospital.add_connection("ICU", "Laboratory")
    hospital.add_connection("Emergency", "Laboratory")

    print("Hospital Route Finder")
    hospital.display_graph()

    print("\nCheck connection:")
    print("Reception to Laboratory:", hospital.is_connected("Reception", "Laboratory"))
    print("Reception to X-Ray:", hospital.is_connected("Reception", "X-Ray"))

    print("\nRoute from Reception to Laboratory:", hospital.find_route("Reception", "Laboratory"))
    print("Route from Pharmacy to ICU:", hospital.find_route("Pharmacy", "ICU"))

    print("\nRunning test cases ... ")
    test_hospital_graph()
```

Code

Output

```
Hospital Route Finder  
Hospital Graph (Adjacency List):  
Reception -> ['Pharmacy', 'ICU']  
Pharmacy -> ['Reception', 'Emergency']  
ICU -> ['Reception', 'Emergency', 'Laboratory']  
Emergency -> ['Pharmacy', 'ICU', 'Laboratory']
```

Observation

This program represents the hospital as a graph where each location is a node and each path is an edge.

It helps us find whether two departments are connected and also find the shortest route between them.

A graph is useful here because hospital locations are connected in different ways, not in a strict hierarchy.

Problem-03

Heap: Emergency Patient Priority

Prompt

Write a python code in which Implement a Priority Queue using a Heap for an emergency department. Patients are treated according to severity: 1 = Critical, 2 = Serious, 3 = Moderate, and 4 = Normal. The queue should support:

1. Add a patient with a priority value.
2. View the highest-priority patient.
3. Remove the highest-priority patient.
4. Display waiting patients.
5. Handle at least one tie in priority appropriately.
6. Test insertion and removal operations.

Observation

A min-heap is ideal because the smallest priority value has the highest urgency.

- Using a sequence counter ensures stable order when two patients share the same priority.
- Removing from the heap always gives the patient with the highest priority first.
- This structure makes emergency treatment decisions fast and efficient.

Code

```
first.py

import heapq

class PatientPriorityQueue:
    def __init__(self):
        self.heap = []
        self.counter = 0

    def add_patient(self, name, priority):
        if priority not in (1, 2, 3, 4):
            raise ValueError("Priority must be 1, 2, 3, or 4.")
        heapq.heappush(self.heap, (priority, self.counter, name))
        self.counter += 1

    def view_highest_priority(self):
        if not self.heap:
            return "No patients waiting."
        priority, _, name = self.heap[0]
        return f"Highest priority patient: {name} (Priority {priority})"

    def remove_highest_priority(self):
        if not self.heap:
            return "No patients waiting."
        priority, _, name = heapq.heappop(self.heap)
        return f"Removed patient: {name} (Priority {priority})"

    def display_waiting_patients(self):
        if not self.heap:
            return "No patients waiting."
        patients = []
        for priority, _, name in self.heap:
            patients.append(f"{name} (Priority {priority})")
        return "Waiting patients: " + ", ".join(patients)

    def __str__(self):
        return self.display_waiting_patients()

# Demo / test logic
if __name__ == "__main__":
    pq = PatientPriorityQueue()

    pq.add_patient("Riya", 3)
    pq.add_patient("Aman", 1)
    pq.add_patient("Neha", 2)
    pq.add_patient("Karan", 1)

    print("Initial queue:")
    print(pq.display_waiting_patients())
    print()

    print(pq.view_highest_priority())
    print(pq.remove_highest_priority())
    print()

    print("Queue after removal:")
    print(pq.display_waiting_patients())
    print()

    print("Next highest priority:")
    print(pq.view_highest_priority())
```

Code

Output

```
Initial queue:
Waiting patients: Aman (Priority 1), Karan (Priority 1), Neha (Priority 2), Riya (Priority 3)

Highest priority patient: Aman (Priority 1)
Removed patient: Aman (Priority 1)

Queue after removal:
Waiting patients: Karan (Priority 1), Riya (Priority 3), Neha (Priority 2)

Next highest priority:
Highest priority patient: Karan (Priority 1)
PS C:\Users\DELL\OneDrive\Desktop\New folder\WTMP LAB>
```

Problem-04

Trie: Medical Word / Drug Search

Prompt

Create a Trie data structure for a hospital application that supports medical word search. Insert medical words, search for an exact word, check whether a prefix exists, and display all words beginning with a given prefix. Example: if the user enters the prefix 'car', the system should suggest matching medical terms such as 'cardiology' and 'cardiac'

Observation

A trie is efficient for prefix-based searching because it shares common characters.

- It performs faster than scanning a list for many prefix queries.
- Searching for an exact word and checking a prefix are both straightforward.
- Autocomplete becomes easy because words can be retrieved from the node of the prefix

Output

```
Medical words inserted:
['cardiology', 'cardiac', 'carcinoma', 'care', 'diagnosis', 'diabetes', 'disease', 'anemia', 'surgery']

Prefix check for 'car': True
Autocomplete for 'car': ['cardiology', 'cardiac', 'carcinoma', 'care']

Search exact word 'cardiac': True
Search exact word 'brain': False

Autocomplete for 'dia': ['diagnosis', 'diabetes']
```

Code

```
first.py

class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end_of_word = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word):
        node = self.root
        for ch in word:
            if ch not in node.children:
                node.children[ch] = TrieNode()
            node = node.children[ch]
        node.is_end_of_word = True

    def search_word(self, word):
        node = self.root
        for ch in word:
            if ch not in node.children:
                return False
            node = node.children[ch]
        return node.is_end_of_word

    def prefix_exists(self, prefix):
        node = self.root
        for ch in prefix:
            if ch not in node.children:
                return False
            node = node.children[ch]
        return True

    def autocomplete(self, prefix):
        node = self.root
        for ch in prefix:
            if ch not in node.children:
                return []
            node = node.children[ch]

        results = []

        def dfs(current_node, current_word):
            if current_node.is_end_of_word:
                results.append(current_word)
            for ch, child in current_node.children.items():
                dfs(child, current_word + ch)

        dfs(node, prefix)
        return results

# Demonstration
if __name__ == "__main__":
    trie = Trie()

    medical_words = [
        "cardiology",
        "cardiac",
        "carcinoma",
        "care",
        "diagnosis",
        "diabetes",
        "disease",
        "anemia",
        "surgery"
    ]

    for word in medical_words:
        trie.insert(word)

    print("Medical words inserted:")
    print(medical_words)
    print()

    prefix = "car"
    print(f"Prefix check for '{prefix}':", trie.prefix_exists(prefix))
    print(f"Autocomplete for '{prefix}':", trie.autocomplete(prefix))
    print()

    search_word = "cardiac"
    print(f"Search exact word '{search_word}':", trie.search_word(search_word))
    print(f"Search exact word 'brain':", trie.search_word("brain"))
    print()

    prefix2 = "dia"
    print(f"Autocomplete for '{prefix2}':", trie.autocomplete(prefix2))
```

Problem-05

AI Decision Challenge: Choose the Correct Structure

Prompt

Write a python code ,For each hospital software scenario, choose the most appropriate non-linear data structure:

Tree, Graph, Heap, or Trie. Explain why it is appropriate, identify the main operation,

state the expected Big-O complexity where relevant, and compare your choice with an AI recommendation.

Observation

- A Tree is best for department hierarchy because it models parent-child relationships.
- A Graph is suitable for room navigation because data is connected by paths.
- A Heap is ideal for priority-based emergency care because the most urgent patient is handled first.
- A Trie is best for autocomplete because it supports fast prefix-based matching.

Code

```
first.py

SCENARIOS = [
    {
        "scenario": "Hospital department hierarchy",
        "choice": "Tree",
        "reason": "Departments are organized in a parent-child hierarchy.",
        "operation": "Insert department and traverse the hierarchy.",
        "complexity": "O(h) for traversal, where h is the height of the tree."
    },
    {
        "scenario": "Finding a route between hospital rooms",
        "choice": "Graph",
        "reason": "Rooms are connected by paths, and routes can have multiple connections.",
        "operation": "Find the shortest path between two rooms.",
        "complexity": "BFS: O(V + E)."
    },
    {
        "scenario": "Emergency patients based on severity",
        "choice": "Heap",
        "reason": "The patient with the highest urgency must be treated first.",
        "operation": "Insert patient and remove the highest-priority patient.",
        "complexity": "Insertion/Removal: O(log n)."
    },
    {
        "scenario": "Autocomplete medical terms",
        "choice": "Trie",
        "reason": "Words share common prefixes, and autocomplete needs fast prefix matching.",
        "operation": "Insert words and search by prefix.",
        "complexity": "Prefix search: O(k), where k is the prefix length."
    },
    {
        "scenario": "Frequently finding the highest-priority task",
        "choice": "Heap",
        "reason": "A priority queue is designed to access the most urgent task quickly.",
        "operation": "Peek or remove the highest-priority task.",
        "complexity": "Peek: O(1), Remove: O(log n)."
    }
]

def print_scenario_answer(item):
    print(f"\nScenario: {item['scenario']}")
    print(f"Choice: {item['choice']}")
    print(f"Reason: {item['reason']}")
    print(f>Main operation: {item['operation']}")
    print(f"Expected complexity: {item['complexity']}")
    print("AI Recommendation: Same choice, because it is the most suitable structure for this problem.")

if __name__ == "__main__":
    print("AI Decision Challenge: Choose the Correct Structure")
    print("Complete answer sheet")

    for item in SCENARIOS:
        print_scenario_answer(item)
```

Code

Output

AI Decision Challenge: Choose the Correct Structure
Complete answer sheet

Scenario: Hospital department hierarchy
Choice: Tree

Reason: Departments are organized in a parent-child hierarchy.
Main operation: Insert department and traverse the hierarchy.
Expected complexity: $O(h)$ for traversal, where h is the height of the tree.
AI Recommendation: Same choice, because it is the most suitable structure for this problem.

Scenario: Finding a route between hospital rooms
Choice: Graph

Reason: Rooms are connected by paths, and routes can have multiple connections.
Main operation: Find the shortest path between two rooms.
Expected complexity: BFS: $O(V + E)$.
AI Recommendation: Same choice, because it is the most suitable structure for this problem.

Scenario: Emergency patients based on severity
Choice: Heap

Reason: The patient with the highest urgency must be treated first.
Main operation: Insert patient and remove the highest-priority patient.
Expected complexity: Insertion/Removal: $O(\log n)$.
AI Recommendation: Same choice, because it is the most suitable structure for this problem.

Scenario: Autocomplete medical terms
Choice: Trie

Reason: Words share common prefixes, and autocomplete needs fast prefix matching.
Main operation: Insert words and search by prefix.
Expected complexity: Prefix search: $O(k)$, where k is the prefix length.
AI Recommendation: Same choice, because it is the most suitable structure for this problem.

Scenario: Frequently finding the highest-priority task
Choice: Heap